Balochistan Procurement Guide
Independent Procurement Knowledge Platform

Database Architecture & ER Design

BPG Platform · Version 1.0 · PostgreSQL · Production reference for backend implementation

Document	**Database Architecture & ERD**
Product	**Balochistan Procurement Guide (BPG)**
Version	**1.0** · derived from approved SRS v1.0
Engine	**PostgreSQL 16** (+ pgvector, full-text search)
Status	**Approved for implementation**
Owner	Muhammad Ajmal Khan — Product Owner

Scope. This document specifies the production database architecture for BPG v1.0: engine selection, the entity-relationship model, a full data dictionary, normalization to 3NF, indexing, full-text + semantic search, security, scalability and backup strategy. It contains no application code and changes no UI or workflow — it implements the approved SRS.

§ Contents

---

1. Architecture Overview

BPG uses a single relational **PostgreSQL** database as the system of record, complemented by object storage for binary files (original PDFs/DOCX/ZIP) and a vector index (pgvector) for AI retrieval. The relational core holds identity & RBAC, a document-centric content model, the contribution/review pipeline, AI conversation history, user engagement (bookmarks, collections, downloads), and full audit/versioning.

- **System of record:** PostgreSQL (normalized, ACID).
- **Binary store:** object storage (e.g. S3-compatible); the DB stores URLs/keys, checksums and metadata only.
- **Search:** native PostgreSQL full-text (`tsvector`/GIN) for keyword + OCR + metadata; `pgvector` for semantic / RAG retrieval. A dedicated engine (OpenSearch) is an optional later swap behind the same schema.
- **Soft deletes & versioning** are first-class; nothing is hard-deleted in v1.0.

---

2. Database Technology Selection

Criterion	PostgreSQL 16	MySQL 8	SQL Server
Full-text search	Native, powerful (tsvector, GIN, ranking)	Basic	Good
Vector / AI (RAG)	**pgvector** — in-database embeddings	None native	Limited / external
JSON / semi-structured	**JSONB** with indexing	JSON (weaker)	JSON (adequate)
Advanced types	Arrays, ranges, ENUM, UUID, GIS	Limited	Moderate
Licensing / cost	**Open source, free**	Open source	Commercial
Scalability & extensions	Excellent (partitioning, replicas, extensions)	Good	Excellent (commercial)

Decision: PostgreSQL 16. It uniquely combines strong relational integrity, native full-text search, JSONB flexibility for evolving metadata, and `pgvector` for AI retrieval — all in one engine. This minimises moving parts for v1.0 (no separate search/vector service required), is open-source (no licensing cost), and scales via read replicas, partitioning and connection pooling. MySQL lacks native vector/strong FTS; SQL Server adds licensing cost without a decisive advantage here.

3. Naming Conventions

- Tables: `snake_case`, plural nouns (`documents`, `ai_messages`). Join tables: `singularA_singularB` (`role_permissions`).
- Primary keys: `id` as `BIGINT GENERATED ALWAYS AS IDENTITY` (or `UUID` for externally-exposed entities).
- Foreign keys: `<referenced_singular>_id` (`document_id`, `user_id`).
- Booleans: `is_` / `has_` prefix. Timestamps: `created_at`, `updated_at`, `deleted_at` (`TIMESTAMPTZ`).
- Indexes: `ix_<table>_<cols>`; unique: `uq_<table>_<cols>`; FKs: `fk_<table>_<ref>`.
- Enumerations stored as Postgres `ENUM` types or lookup tables where values evolve.

4. ER Diagram

Core entity-relationship model (document-centric, with RBAC, contribution pipeline, AI and engagement clusters). Full cardinalities are listed in §7.

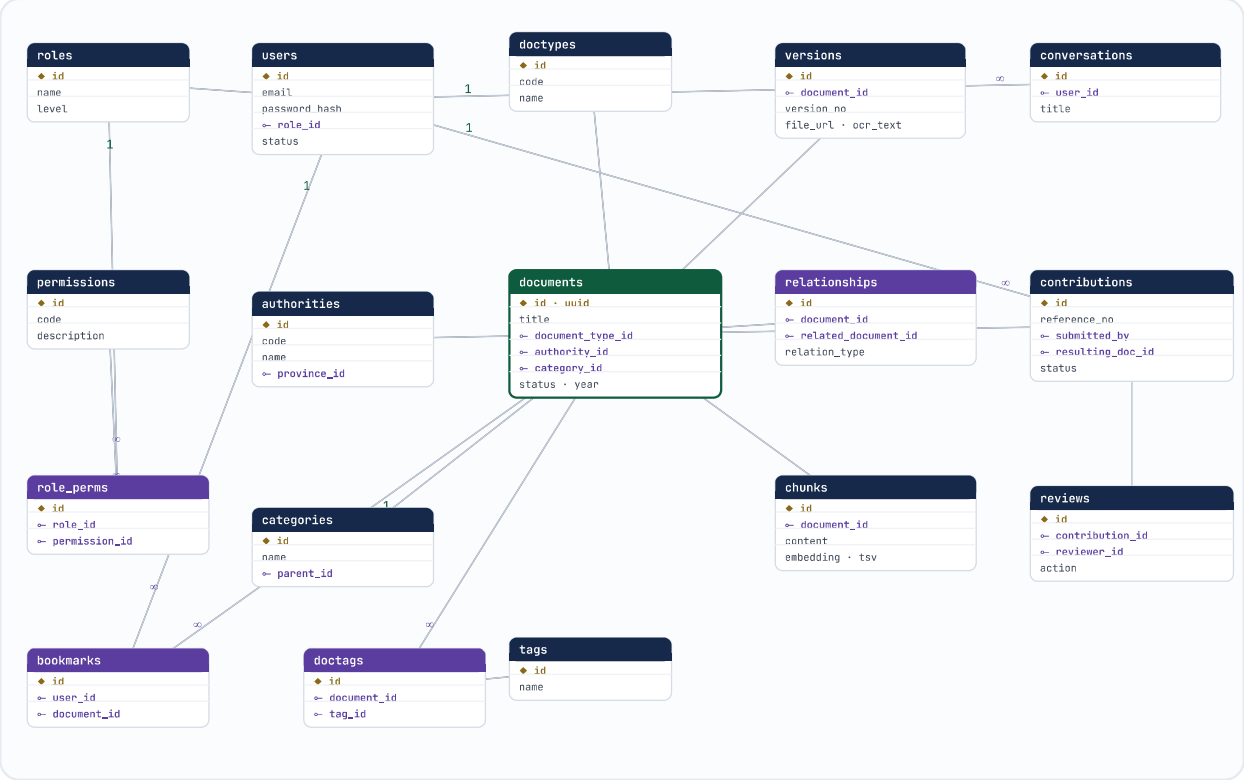

Figure 1 — BPG core ER diagram. ◆ PK · FK · lines show 1 → ∞ (one-to-many) and ∞—∞ (many-to-many via join tables).

Identity/RBAC: users, roles, permissions, role_permissions

Content: documents, document_types, authorities, categories, versions, tags **Pipeline:** contributions, contribution_reviews

AI: conversations, messages, citations, document_chunks

---

5. Schema Design & Normalization (3NF)

The model is normalized to **Third Normal Form**: every non-key attribute depends on the key, the whole key and nothing but the key. Lookups (authorities, categories, document_types, provinces, tags) are separated to remove repeating groups and transitive dependencies.

Polymorphic content model — one `documents` table, not twenty

The SRS lists many content types (Acts, Rules, Regulations, Guidelines, Notifications, Circulars, Office Orders, PRC Decisions, Court Judgments, Research, FIDIC, NHA, PEC, PSDP, CSR…). Creating a separate near-identical table per type would duplicate 90% of columns and violate DRY/normalization. Instead:

- A single `documents` table holds all common attributes, typed by `document_type_id` → `document_types`.
- Type-specific fields that genuinely differ (e.g. a judgment's *court*, *case number*, *precedent value*; a CSR's *region*/*rate period*) live in a flexible `attributes JSONB` column and/or thin extension tables (`judgment_details`, `csr_details`) where strong typing/indexing is needed.
- Engineering libraries (FIDIC/NHA/PEC/PSDP/CSR) are modelled as `authorities` + `categories` on the same `documents` table, not as separate tables.

> **Why this matters.** One content model means search, AI retrieval, bookmarking, versioning, permissions and audit are written once and apply to every document type — and new types (office orders, gazette notifications, international resources) are added as a `document_types` row with zero schema migration.

---

6. Data Dictionary

> **Common columns (on every table unless noted).** `id` PK; `created_at TIMESTAMPTZ NOT NULL DEFAULT now()`; `updated_at TIMESTAMPTZ NOT NULL DEFAULT now()` (trigger-maintained); `deleted_at TIMESTAMPTZ NULL` (soft delete — `NULL` = active). These are omitted from the per-table lists below to avoid repetition.

6.1 Identity & Access Control

`users`

Purpose: all accounts (guest activity is anonymous/unpersisted; registered users and staff live here).

Column	Type	Constraints / notes
`id`	BIGINT	PK, identity
`email`	CITEXT	NOT NULL, **UNIQUE**
`password_hash`	TEXT	NOT NULL (argon2id/bcrypt); never plaintext
`full_name`	TEXT	NULL
`role_id`	BIGINT	FK → roles(id), NOT NULL, default = "registered"
`status`	ENUM	active / suspended / pending — default 'pending'
`email_verified_at`	TIMESTAMPTZ	NULL
`last_login_at`	TIMESTAMPTZ	NULL
`settings`	JSONB	NOT NULL DEFAULT '{}' (prefs, language)

Indexes: `uq_users_email`; `ix_users_role_id`; partial `ix_users_active (deleted_at IS NULL)`.

`roles` · `permissions` · `role_permissions`

Table	Key columns	Notes
`roles`	**`id`**, `name` (UNIQUE), `level` SMALLINT	guest, registered, contributor, moderator, admin, super_admin
`permissions`	**`id`**, `code` (UNIQUE), `description`	e.g. `document.publish`, `user.manage`
`role_permissions`	**`id`**, **`role_id`**, **`permission_id`**	**M:N** join; **UNIQUE**(role_id, permission_id)

6.2 Taxonomy & Lookups

Table	Purpose	Key columns
`provinces`	Jurisdictions	**`id`**, `name` (Balochistan, Federal, Punjab, Sindh, KP, ...)
`authorities`	Issuing bodies	**`id`**, `code` (UNIQUE), `name`, **`province_id`**, `website_url`
`categories`	Browse taxonomy (self-referential tree)	**`id`**, `name`, `slug` (UNIQUE), **`parent_id`** → categories
`document_types`	Act, Rules, Regulation, Guideline, Notification, Circular, Office Order, PRC, Judgment, Research, SBD...	**`id`**, `code` (UNIQUE), `name`
`tags`	Free keywords	**`id`**, `name` (UNIQUE), `slug`

6.3 Documents, Versions & Relationships

`documents`

Purpose: the unified record for every published item of any type.

Column	Type	Constraints / notes
id	BIGINT	PK
uuid	UUID	UNIQUE, default gen_random_uuid() (public URL key)
title	TEXT	NOT NULL
document_type_id	BIGINT	FK → document_types, NOT NULL
authority_id	BIGINT	FK → authorities, NOT NULL
category_id	BIGINT	FK → categories, NULL
province_id	BIGINT	FK → provinces, NULL
document_number	TEXT	NULL (gazette/notification no.)
year	SMALLINT	NULL
language	ENUM	en / ur / bilingual — default 'en'
status	ENUM	in_force / amended / repealed / superseded / draft / historical
effective_date	DATE	NULL
summary	TEXT	NULL (AI/editorial abstract)
attributes	JSONB	NOT NULL DEFAULT '{}' (type-specific fields)
search_tsv	TSVECTOR	generated (title+summary+ocr) — GIN indexed
current_version_id	BIGINT	FK → document_versions (latest)
published_by	BIGINT	FK → users, NULL
published_at	TIMESTAMPTZ	NULL

Indexes: `uq_documents_uuid`; `ix_documents_type / authority / category / status / year`; GIN on `search_tsv` and on `attributes`.

`document_versions`

Column	Type	Notes
`id`	BIGINT	PK
`document_id`	BIGINT	FK → documents, NOT NULL
`version_no`	TEXT	e.g. "2.0"; **UNIQUE**(document_id, version_no)
`file_url`	TEXT	object-storage key (original)
`file_type`	ENUM	pdf / docx / zip
`file_size`	BIGINT	bytes
`checksum_sha256`	TEXT	integrity / dedup
`page_count`	INT	NULL
`ocr_text`	TEXT	extracted text layer
`ocr_confidence`	NUMERIC(5,2)	NULL
`change_note`	TEXT	amendment description

`document_relationships` · `document_tags` · `document_details (extensions)`

Table	Key columns	Notes
`document_relationships`	`document_id`, `related_document_id`, `relation_type`	self-M:N (amends, supersedes, references, related); UNIQUE triple
`document_tags`	`document_id`, `tag_id`	**M:N**; UNIQUE(document_id, tag_id)
`judgment_details`	`document_id` (PK/FK), `court`, `case_no`, `precedent_value`	1:1 extension (binding/persuasive/reference)
`csr_details`	`document_id`, `region`, `rate_period`	1:1 extension (engineering rates)

6.4 Search & AI Retrieval

`document_chunks`

Purpose: chunked text + embeddings for RAG / semantic search.

Column	Type	Notes
id	BIGINT	PK
document_id	BIGINT	FK → documents, NOT NULL
version_id	BIGINT	FK → document_versions
chunk_index	INT	order within document
content	TEXT	chunk text
embedding	VECTOR(1536)	pgvector; HNSW/IVFFlat index
tsv	TSVECTOR	GIN (hybrid keyword+semantic)

ai_conversations · ai_messages · ai_citations

Table	Key columns	Notes
ai_conversations	id, user_id, title	FK user nullable (anonymous sessions optional)
ai_messages	id, conversation_id, role, content, reading_mode, confidence	role = user/assistant; 1:N from conversation
ai_citations	id, message_id, document_id, chunk_id, locator	which sources grounded the answer (M:N message↔document)

6.5 Community Contribution & Review

Table	Key columns	Notes
contributions	id, reference_no (UNIQUE), submitted_by (NULL=anon), is_anonymous, submitted metadata, file_url, status	status ENUM: pending / under_review / approved / rejected / need_info / duplicate
contribution_reviews	id, contribution_id, reviewer_id, action, note, resulting_document_id	action ENUM: approve/reject/merge/archive/publish/request_info; 1:N audit of decisions
duplicate_flags	contribution_id, candidate_document_id, similarity	detection results

6.6 User Engagement

Table	Key columns	Notes
bookmarks	**user_id**, **document_id**, anchor	UNIQUE(user_id, document_id, anchor)
collections	**id**, **user_id**, name, is_shared, share_token	named sets
collection_items	**collection_id**, **document_id**	**M:N**; UNIQUE(collection_id, document_id)
downloads	**user_id** (NULL), **document_id**, format, ip_hash	1 row per download (analytics)
search_history	**user_id** (NULL), query, filters JSONB, result_count	powers recent/popular
saved_searches	**user_id**, name, query, filters JSONB, alert	future alerts
notifications	**id**, **user_id**, type, payload JSONB, read_at	in-app + email

6.7 System, Audit & Future

Table	Key columns	Notes
activity_logs	**user_id**, action, entity_type, entity_id, meta JSONB	operational feed (recent activity)
audit_logs	**actor_id**, action, entity_type, entity_id, before JSONB, after JSONB, ip	**immutable**, append-only; privileged actions
system_settings	key (PK), value JSONB, updated_by	config (thresholds, AI/version)
document_versions	(see 6.3)	serves as the version-history table
comments (future)	**user_id**, **document_id**, **parent_id**, body, status	threaded; moderation
api_keys (future)	**user_id**, key_hash, scopes, last_used_at, revoked_at	hashed; never store raw key

---

7. Relationships Explained

- **users** → **roles** (N:1): each user has one primary role. **roles** ↔ **permissions** (M:N) via `role_permissions` — RBAC.
- **documents** → **document_types / authorities / categories / provinces** (each N:1): classification via lookups (3NF).
- **documents** → **document_versions** (1:N); `documents.current_version_id` points to the latest (1:1 convenience).
- **documents** ↔ **documents** (M:N) via `document_relationships` (amends/supersedes/references).
- **documents** ↔ **tags** (M:N) via `document_tags`.

- **documents** → **judgment_details / csr_details** (1:1) type extensions.
- **documents** → **document_chunks** (1:N) for embeddings/FTS.
- **users** → **ai_conversations** → **ai_messages** (1:N each); **ai_messages** ↔ **documents** (M:N) via `ai_citations`.
- **users** → **contributions** → **contribution_reviews** (1:N each); an approved contribution yields a `documents` row (`resulting_document_id`).
- **users** ↔ **documents** via `bookmarks` (M:N); **collections** ↔ **documents** (M:N) via `collection_items`.
- **users** → **downloads / search_history / saved_searches / notifications / audit_logs** (1:N).

---

8. Index Strategy

Goal	Indexes
Keyword / OCR / metadata search	GIN on `documents.search_tsv` and `document_chunks.tsv`; GIN on `documents.attributes` (JSONB).
Semantic / AI retrieval (RAG)	HNSW (or IVFFlat) on `document_chunks.embedding` (cosine).
Repository filtering	B-tree on `document_type_id`, `authority_id`, `category_id`, `province_id`, `status`, `year`; composite `(status, document_type_id, year DESC)`.
Sorting	B-tree on `published_at DESC`, `year`, `title`.
Lookups / FKs	Index every FK column; UNIQUE on natural keys (email, codes, slugs, reference_no, uuid).
Soft-delete & hot rows	Partial indexes `WHERE deleted_at IS NULL`; queue tables indexed on `status`.

---

9. Full-Text & Semantic Search

- **Keyword:** `tsvector` columns (weighted: title > summary > OCR body) with GIN indexes and `ts_rank` ordering.
- **OCR text:** stored on `document_versions.ocr_text` , folded into `documents.search_tsv` on publish.
- **Metadata:** JSONB `attributes` with GIN for typed filters.
- **Semantic / RAG:** `document_chunks.embedding` (pgvector) queried by cosine distance; results fused with keyword hits (hybrid search) and passed to the AI with their `document_id` / `chunk_id` for citation.
- **Future swap:** the same chunk/embedding model maps cleanly to OpenSearch/pgvector-at-scale without schema change.

10. Security

- **Password storage:** argon2id (or bcrypt cost ≥ 12) in `password_hash`; plaintext never stored or logged.
- **RBAC:** enforced via `roles` / `permissions` / `role_permissions`; application checks permission codes server-side; optional Postgres Row-Level Security for owner-scoped tables (bookmarks, collections, conversations).
- **Audit logs:** `audit_logs` is append-only/immutable (no UPDATE/DELETE grants), capturing actor, action, before/after JSONB and IP for every privileged operation.
- **Encryption:** TLS in transit; at-rest encryption on DB and object storage; sensitive secrets (API keys) stored only as hashes.
- **Soft deletes:** `deleted_at` everywhere; reads filter it out; nothing destructive in v1.0.
- **Version history:** `document_versions` preserves every revision; amendments add versions rather than overwriting.

---

11. Scalability

- **100k+ documents / users:** BIGINT keys, targeted indexes, list virtualisation in the app, and table partitioning for high-growth logs (`activity_logs`, `downloads`, `search_history`) by month.
- **Millions of AI queries:** `ai_messages` / `document_chunks` partition-friendly; read replicas for search/AI retrieval; connection pooling (PgBouncer).
- **Future APIs / mobile:** UUID public keys, stable schema, `api_keys` table ready; the same DB serves web, mobile and API.
- **International expansion:** add `provinces` / `authorities` / `document_types` rows and a `country` dimension — no structural change.

---

12. Backup & Disaster Recovery

- **Database:** daily logical (`pg_dump`) + continuous WAL archiving for point-in-time recovery (PITR); encrypted, off-site, retained per policy.
- **Document storage:** versioned object storage with cross-region replication; DB stores checksums to detect corruption.
- **Version recovery:** document-level rollback via `document_versions`; row-level recovery via PITR.
- **DR targets:** RPO ≤ 24h (PITR ≤ minutes for the DB), RTO ≤ 8h; restore drills tested quarterly with a documented runbook.

13. Best Practices & Future Expansion

- All schema changes via versioned migrations (Flyway/Liquibase/native); no manual production edits.
- Foreign keys enforced with sensible `ON DELETE` (RESTRICT for content, CASCADE for owned children like messages/items).
- `updated_at` maintained by trigger; ENUMs for closed sets, lookup tables for evolving sets.
- Keep JSONB for genuinely variable attributes only; promote frequently-queried JSON keys to typed columns/extension tables.
- **Future-ready (no major migration):** `comments` and `api_keys` tables; `country` dimension; new `document_types`; OpenSearch swap behind the chunk model; partition rollout for logs.

> **Approval.** This database architecture implements the approved BPG SRS v1.0 in Third Normal Form on PostgreSQL 16, with full-text + vector search, RBAC, soft deletes, versioning and audit. It is ready for direct implementation by a senior backend team and accommodates the v1.1+ roadmap without structural change.